



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SEMESTER 2

2025/2026

SECP3843-02 SPECIAL TOPIC IN DATA ENGINEERING

Tutorial 3 - AI Algorithm

Made by:

NAME	MATRIC NUMBER
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

LECTURER: DR. ARYATI BINTI BAKRI

Table of Content

1.0 Image Classification.....	3
1.1 Applications of Image Classification.....	3
2.0 Convolutional Neural Network (CNN).....	3
2.1 Key Components of a CNN.....	4
2.2 Evaluation of CNN.....	4
3.0 Steps Hands-On in Python.....	6
3.1 Step 1: Importing Libraries.....	6
3.2 Step 2: Data Loading and Label Flattening.....	6
3.3 Step 3: Sample Plotting.....	7
3.4 Step 4: Data Normalization.....	8
3.5 Step 5: Building the ANN model.....	8
3.6 Step 6: Compiling and Training the ANN model.....	9
3.7 Step 7: ANN Predictions and Evaluations.....	9
3.8 Step 8: Building the CNN model.....	11
3.9 Step 9: Compiling and Training the CNN model.....	11
3.10 Step 10: Evaluating the CNN model.....	12
4.0 Weakness of the Current CNN model.....	13
5.0 How to enhance CNN model.....	13
5.1 Data augmentation.....	14
5.2 Architectural Tuning and Addition of a Deeper Convolutional Layer.....	14
6.0 Evaluation based on CNN and new_CNN model.....	16
7.0 Results and Discussions.....	19
9.0 Reflections.....	20
References.....	22

1.0 Image Classification

Image classification is a fundamental task in computer vision in which a trained model assigns a predefined category label to an input image based on its visual content. Given a raw image as input, the model analyses pixel-level patterns and outputs the most probable class from a fixed set of categories (LeCun et al., 2015) . In supervised learning, the model learns this mapping by being exposed to thousands of labelled examples, progressively adjusting its internal parameters to minimise prediction error until it can reliably distinguish one class from another.

The CIFAR-10 dataset used in this tutorial (Krizhevsky et al., 2009) is one of the most widely used benchmarks for image classification research. It consists of 60,000 colour images of size 32x32 pixels distributed evenly across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

1.1 Applications of Image Classification

Image classification underpins a wide range of real-world applications across many industries :

- Healthcare: Automated diagnosis of diseases from medical images such as X-rays, MRI scans, and histopathology slides
- Autonomous Vehicles: Identifying road signs, pedestrians, traffic lights, and lane markings in real time.
- Retail and E-Commerce: Automatic product tagging, visual search, and inventory management.
- Security and Surveillance: Face recognition, intrusion detection, and anomaly identification.
- Agriculture: Detecting crop diseases and pests from drone or satellite imagery.

2.0 Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a class of deep learning model specifically designed to process structured grid data such as images. CNNs automatically learn spatial hierarchies of

features directly from raw pixel data without any manual feature engineering (LeCun et al., 2015). Unlike traditional fully connected networks that treat input images as flat vectors and lose all spatial information, CNNs exploit the spatial structure of images by applying shared learnable filters across the entire input, a property that dramatically reduces the number of parameters and improves generalisation.

CNNs are trained using backpropagation and gradient descent. The Adam optimiser, introduced by Kingma and Ba (2015), is commonly used due to its adaptive learning rate, which typically results in faster and more stable convergence compared to vanilla stochastic gradient descent.

2.1 Key Components of a CNN

Convolutional Layer: The core building block of a CNN. It applies a set of learnable filters that slide across the input image, computing dot products between filter weights and local input patches to produce feature maps. Each filter detects a specific low-level feature such as horizontal edges, vertical edges, or colour gradients. Common kernel sizes are 3x3 and 5x5.

Activation Function (ReLU): After each convolution, a Rectified Linear Unit activation is applied: $f(x) = \max(0, x)$. This introduces non-linearity, enabling the network to learn complex decision boundaries. ReLU is preferred over sigmoid or tanh because it mitigates the vanishing gradient problem and speeds up training.

Pooling Layer: Pooling layers perform spatial downsampling to reduce the dimensions of feature maps. Max Pooling takes the maximum value from a defined window (e.g., 2x2 with stride 2), halving the spatial dimensions. This reduces computational cost and provides a degree of translation invariance (LeCun et al., 2015).

2.2 Evaluation of CNN

To benchmark the performance of a CNN model during training and testing, several core statistical metrics are utilized (Grandini et al., 2020):

- (a) **Accuracy:** The ratio of correctly predicted images to the total dataset size. While intuitive, accuracy can be misleading if evaluated on highly imbalanced datasets.

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{TP} + \text{TN} + \text{False Positives (FP)} + \text{False Negatives (FN)}}$$

Figure 2.1 : Formula of Accuracy

- (b) **Loss (Cross-Entropy):** A metric indicating how far the model's predicted probability distribution deviates from the true target label. For multi-class image classification, Categorical Cross-Entropy is standard:

$$\text{Loss} = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

where y_i is the true binary label and $\hat{y}_i$ is the predicted probability.

Figure 2.2 : Formula of Loss

- (c) **Precision:** The proportion of positive identifications that were actually correct. This is critical in scenarios where false positives carry high costs.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Figure 2.3 : Formula of Precision

3.0 Steps Hands-On in Python

This section presents a step-by-step documentation of the implementation of image classification using both an Artificial Neural Network (ANN) and a Convolutional Neural Network (CNN) in Python, executed through Google Colab. The dataset used throughout this implementation is the CIFAR-10 dataset, which consists of 60,000 colour images distributed across 10 distinct classes.

3.1 Step 1: Importing Libraries

To build our deep learning pipeline, we use TensorFlow and Keras for model construction, NumPy for multi-dimensional matrix operations, and Matplotlib for visual diagnostics. Figure 3.1 illustrates the coding used to import libraries.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 3.1: Import Libraries

3.2 Step 2: Data Loading and Label Flattening

Figure 3.2 demonstrates the sequence for loading and structurally verifying the CIFAR-10 dataset using TensorFlow/Keras, alongside a critical array reshaping operation required for model compatibility. Initially, the dataset is loaded into memory, splitting into training and testing features (X_{train} , X_{test}) and their corresponding categorical target matrices (y_{train} , y_{test}). When the properties $X_{\text{test}}.shape$, $X_{\text{train}}.shape$, and $y_{\text{train}}.shape$ are verified, they reveal that the raw target arrays are loaded as 2D column vectors (nested lists of single elements, as seen in the first $y_{\text{train}}[:5]$ slice). To prevent indexing errors during data visualization and to ensure direct compatibility with the baseline model's `sparse_categorical_crossentropy` loss function, the `.reshape(-1,)` method is applied to both y_{train} and y_{test} . This process flattens the 2D column vectors into simple, unnested 1D arrays, which is verified by the final $y_{\text{train}}[:5]$ slice that outputs a clean, sequential integer array suitable for flawless pipeline processing.

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

X_test.shape
Show hidden output

X_train.shape
... Show hidden output

y_train.shape
Show hidden output

y_train[:5]
Show hidden output

y_train = y_train.reshape(-1,)
y_train[:5]
Show hidden output

y_test = y_test.reshape(-1,)
```

Figure 3.2 : Data Loading and Reshaping

3.3 Step 3: Sample Plotting

Figure 3.3 illustrates the exploratory data analysis and sample plotting phase, where a custom helper function (`plot_sample`) is implemented to visually inspect data integrity and verify categorical array alignments. The function utilizes Matplotlib to render the 2D pixel intensity layout of a specified image matrix while programmatically mapping the sample's flat target integer index directly to its string equivalent in the predefined class list. By calling the function on training index position 5, a low-resolution 32 x 32 pixel color image is visually outputted alongside its correctly aligned structural label, "automobile". Running this diagnostic step verifies that the data loading pipeline is fully intact, the image channels are properly mapped, and the categorical textual mappings perfectly sync with the numerical label markers before training begins.

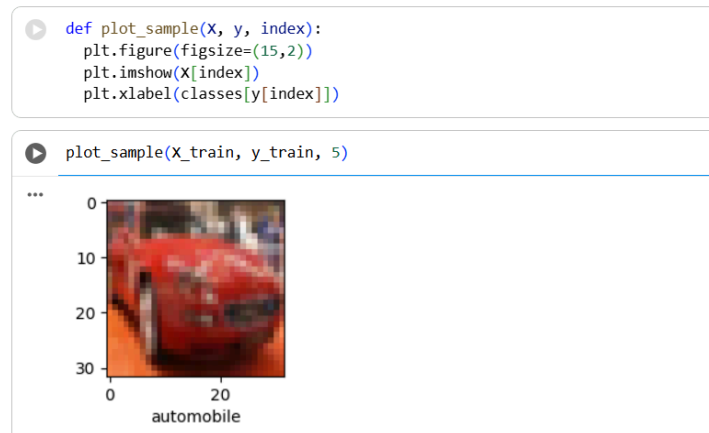


Figure 3.3 : Sample Plotting

3.4 Step 4: Data Normalization

This step performs feature scaling on the dataset by dividing every raw pixel intensity value in the training (X_{train}) and testing (X_{test}) matrices by 255.0. Since raw image pixels are stored as integers ranging from 0 (black) to 255 (white), this mathematical operation rescales the entire feature space down into a standard bounding range between 0.0 and 1.0. Normalization is a critical requirement in deep learning because it prevents gradients from destabilizing or exploding when large integer inputs are passed through the network weights. Uniformly scaling the features ensures smoother weight updates during backpropagation, minimizes computational overhead, and helps the optimization algorithm converge significantly faster to a stable minimum loss

```
X_train = X_train / 255.0  
X_test = X_test / 255.0
```

Figure 3.4: Data Normalization

3.5 Step 5: Building the ANN model

The first model constructed is a baseline Artificial Neural Network (ANN) using the Keras Sequential API. As illustrated in Figure 3.5, the model architecture begins with a Flatten layer that reshapes the $32 \times 32 \times 3$ input images into a one-dimensional vector of 3,072 values. This is

followed by two fully connected Dense hidden layers containing 3,000 and 1,000 neurons respectively, both utilising the Rectified Linear Unit (ReLU) activation function to introduce non-linearity into the learning process. A final Dense output layer comprising 10 neurons with a Softmax activation function is added to produce a probability distribution of CIFAR-10 classes, where the class with the highest probability is selected as the prediction.

```
ann = models.Sequential([
    layers.Flatten(input_shape=(32,32,3)),
    layers.Dense(3000, activation='relu'),
    layers.Dense(1000, activation='relu'),
    layers.Dense(10, activation='softmax')
])

ann.compile(optimizer='sgd',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

Figure 3.5: ANN model architecture code

3.6 Step 6: Compiling and Training the ANN model

Following the construction of the ANN model, the model was compiled using Stochastic Gradient Descent (SGD) as the optimiser and sparse categorical crossentropy as the loss function, which is suitable for multi-class classification problems with integer-encoded class labels. The model was subsequently trained on the CIFAR-10 training dataset over five epochs, as shown in Figure 3.6. The training process iterates over the entire training set in each epoch, adjusting the model's weights based on the computed gradients to minimise the loss.

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequen
super().__init__(**kwargs)
Epoch 1/5
1563/1563 — 126s 80ms/step - accuracy: 0.3524 - loss: 1.8158
Epoch 2/5
1563/1563 — 140s 79ms/step - accuracy: 0.4251 - loss: 1.6269
Epoch 3/5
1563/1563 — 125s 80ms/step - accuracy: 0.4545 - loss: 1.5443
Epoch 4/5
1563/1563 — 142s 80ms/step - accuracy: 0.4771 - loss: 1.4831
Epoch 5/5
1563/1563 — 149s 84ms/step - accuracy: 0.4958 - loss: 1.4334
<keras.src.callbacks.history.History at 0x7b6c905daf60>
```

Figure 3.6: ANN model compilation and training output showing accuracy per epoch

3.7 Step 7: ANN Predictions and Evaluations

Upon completion of training, predictions were generated on the test dataset using `ann.predict(X_test)`. The predicted class for each test image was determined by extracting the index with the highest probability using `np.argmax()`. A classification report was subsequently generated using Scikit-learn's `classification_report` function, displaying precision, recall, and

F1-score for each of the 10 classes, as presented in Figure 3.7.1. Additionally, a confusion matrix was visualised as a heatmap using the Seaborn library, as shown in Figure 3.7.2, to provide a visual representation of the model's classification performance across all classes.

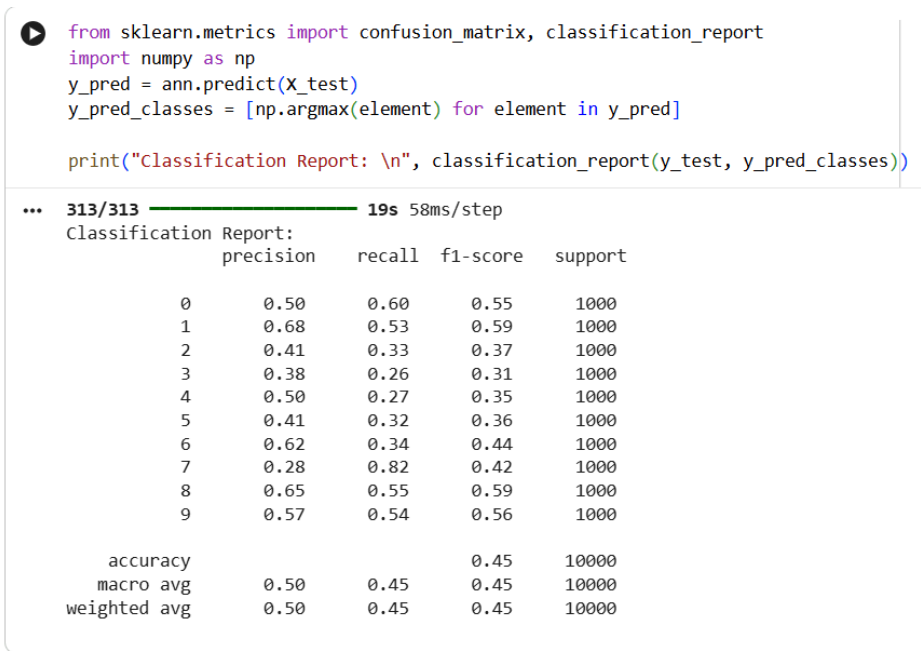


Figure 3.7.1: Classification Report of the ANN Model

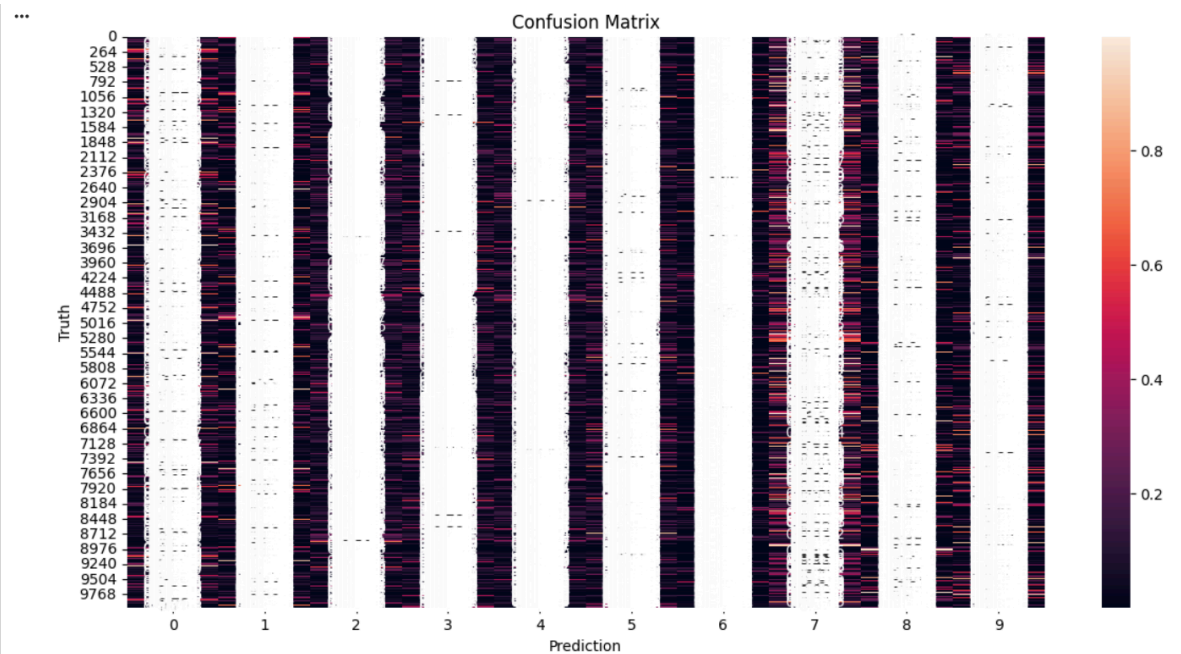


Figure 3.7.2: Confusion matrix heatmap of ANN model predictions

As evidenced by the classification report in Figure 3.7.1 and the confusion matrix in Figure 3.7.2, the ANN model yielded a limited test accuracy of approximately 49%. This result indicates that a fully connected neural network architecture is insufficient for effectively capturing the spatial and hierarchical features present in image data, thereby motivating the adoption of a Convolutional Neural Network.

3.8 Step 8: Building the CNN model

A Convolutional Neural Network (CNN) was subsequently constructed to leverage the spatial structure of image data. As depicted in Figure 3.4.1, the CNN architecture comprises two convolutional blocks, each consisting of a Conv2D layer followed by a MaxPooling2D layer. The first Conv2D layer applies 32 filters of size 3×3 to detect low-level features such as edges and colours, while the second Conv2D layer applies 64 filters of the same kernel size to extract higher-level features. Each MaxPooling2D layer with a pool size of 2×2 reduces the spatial dimensions of the feature maps by half, thereby decreasing computational complexity while retaining essential feature information. A Flatten layer then converts the two-dimensional feature maps into a one-dimensional vector, which is passed through a Dense layer of 64 neurons with ReLU activation before the final output Dense layer of 10 neurons with Softmax activation.

```
cn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 3.8.1: CNN model architecture code

3.9 Step 9: Compiling and Training the CNN model

The CNN model was compiled using the Adam optimiser, which adapts the learning rate dynamically during training and is widely regarded as more effective than SGD for deep learning tasks. The loss function remained sparse categorical crossentropy, consistent with the ANN

model. The CNN was trained over 10 epochs, as shown in Figure 3.9.1, allowing the model additional iterations to refine its learned feature representations compared to the ANN's five epochs.

```
cnn.fit(X_train, y_train, epochs=10)
```

Epoch 1/10	1563/1563	58s	36ms/step	- accuracy: 0.4777	- loss: 1.4603
Epoch 2/10	1563/1563	55s	35ms/step	- accuracy: 0.6163	- loss: 1.1019
Epoch 3/10	1563/1563	83s	36ms/step	- accuracy: 0.6637	- loss: 0.9674
Epoch 4/10	1563/1563	55s	35ms/step	- accuracy: 0.6947	- loss: 0.8854
Epoch 5/10	1563/1563	55s	35ms/step	- accuracy: 0.7166	- loss: 0.8206
Epoch 6/10	1563/1563	56s	36ms/step	- accuracy: 0.7341	- loss: 0.7733
Epoch 7/10	1563/1563	56s	36ms/step	- accuracy: 0.7494	- loss: 0.7264
Epoch 8/10	1563/1563	83s	36ms/step	- accuracy: 0.7631	- loss: 0.6882
Epoch 9/10	1563/1563	56s	36ms/step	- accuracy: 0.7742	- loss: 0.6489
Epoch 10/10	1563/1563	85s	38ms/step	- accuracy: 0.7841	- loss: 0.6175

<keras.src.callbacks.history.History at 0x7b6b44eee0c0>

Figure 3.9.1: CNN model training output

3.10 Step 10: Evaluating the CNN model

The trained CNN model was evaluated on the held-out test dataset using `cnn.evaluate(X_test, y_test)`, which returns the test loss and test accuracy. As shown in Figure 3.10.1, the CNN model achieved a test accuracy of approximately 70%, representing a significant improvement of approximately 21 percentage points over the baseline ANN model. This improvement demonstrates the effectiveness of convolutional and pooling layers in automatically extracting discriminative spatial features from image data without manual feature engineering.

```
cnn.evaluate(X_test, y_test)
```

313/313	4s	11ms/step	- accuracy: 0.7032	- loss: 0.9057
---------	----	-----------	--------------------	----------------

[0.9056682586669922, 0.7031999826431274]

Figure 3.10.1: CNN model evaluation result

4.0 Weakness of the Current CNN model

Despite achieving a test accuracy of approximately 70% on the CIFAR-10 dataset, the current CNN model exhibits several notable limitations that constrain its performance.

First, the model is trained for only 10 epochs, which may be insufficient for the model to fully converge, particularly given the complexity and diversity of the CIFAR-10 dataset comprising 60,000 images across 10 distinct classes. As observed in Figure 3.5.1, the training accuracy continues to improve towards the final epochs, suggesting that the model has not yet reached a performance plateau and would likely benefit from additional training iterations.

Second, the absence of data augmentation techniques such as random horizontal flipping, random cropping, and brightness adjustment limits the model's exposure to varied representations of each image class. As a result, the model may struggle to generalise to unseen variations of images during inference, leading to reduced test accuracy.

Third, the current architecture does not incorporate dropout layers, which are a commonly used regularisation technique that randomly deactivates a fraction of neurons during training to prevent the network from over-relying on specific features. Without dropout, the model is more susceptible to overfitting, particularly as model depth increases.

Fourth, the model utilises only two convolutional layers with relatively small filter counts of 32 and 64, respectively. This relatively shallow architecture may restrict the model's capacity to learn complex, high-level abstract representations that are critical for accurately distinguishing between visually similar classes such as cats, dogs, and deer in the CIFAR-10 dataset.

These weaknesses collectively highlight the need for architectural enhancements and training improvements, which are addressed in the subsequent section through the implementation of an enhanced CNN model.

5.0 How to enhance CNN model

The original CNN model provides a functional baseline for image classification. However, it has certain limitations that reduce its overall accuracy. Three enhancement strategies were applied to address these limitations and produce an improved model, referred to as `new_CNN`. The

strategies are: data augmentation, architectural tuning, and the addition of a deeper convolutional layer.

5.1 Data augmentation

Data augmentation increases the variety of training data by applying small, random changes (like flipping) to images without changing their labels. This technique is necessary because the original CNN trains on the same 50,000 images in every epoch. Over time, the model begins to memorise specific pixel patterns rather than learning the general features of each class. This problem is known as overfitting. Augmentation addresses this by ensuring the model encounters a slightly different version of each image in every training round.

The augmentation pipeline was configured using the `ImageDataGenerator` class from Keras as shown in Figure 5.1. Each parameter applies a transformation of no more than 10%. ‘horizontal_flip’ randomly mirrors the image along the vertical axis. The ‘width_shift_range’ and ‘height_shift_range’ shift the image within the frame without changing its content. Moreover, ‘zoom_range’ scales the image slightly inward or outward.

```
datagen = ImageDataGenerator(  
    horizontal_flip = True,      # randomly flip images left-right  
    width_shift_range = 0.1,    # randomly shift images horizontally  
    height_shift_range = 0.1,  # randomly shift images vertically  
    zoom_range = 0.1           # randomly zoom in/out  
)  
  
datagen.fit(X_train)  
print('Augmentation ready!')
```

Figure 5.1: ImageDataGenerator in augmentation

5.2 Architectural Tuning and Addition of a Deeper Convolutional Layer

In addition to data augmentation, four structural changes were made to the model architecture to improve its learning ability and reduce overfitting. Structural changes includes:

- **BatchNormalization:** Inserted after each convolutional layer. This technique normalises the values that pass through the layer to keep them within a consistent range. Stable values allow the model to train faster and reduce the risk of the learning process becoming unstable.

- Dropout (rate = 0.4): Added before the final output layer. During each training step, 40% of the connections are randomly disabled. This prevents the model from becoming too dependent on specific pathways and encourages it to learn more general patterns which reduces overfitting.
- Larger Dense layer: The fully connected hidden layer was increased from 64 units to 128 units. This provides the model with a greater capacity to combine and interpret the features extracted by the convolutional layers before producing a final classification.
- Same Padding: The padding setting in all convolutional layers was changed to 'same'. With the same padding, the convolution covers the entire image including the edges in which preserves spatial information that would otherwise be lost.

In addition, a third block was added to new_CNN to increase the number of filters to 128. Each block in the network is responsible for learning progressively more complex visual features:

- Block 1 (32 filters): Detects basic, low-level features such as edges, lines and colour gradients.
- Block 2 (64 filters): Combines the basic features from Block 1 into shapes and textures.
- Block 3 (128 filters) (new): Combines shapes and textures into high-level object features, such as the outline of a wheel, the pattern of fur or the silhouette of a wing.

This progressive increase in filter count follows a standard deep learning design principle: early layers capture simple and general patterns while later layers capture complex, class-specific representations. The complete new_CNN model is defined as in Figure 5.2.

```

new_CNN = models.Sequential([

    # --- Block 1 ---
    layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32,32,3)),
    layers.BatchNormalization(),          # NEW: stabilise training
    layers.MaxPooling2D((2,2)),

    # --- Block 2 ---
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),

    # --- Block 3 (NEW deeper layer) ---
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),

    # --- Classifier head ---
    layers.Flatten(),
    layers.Dense(128, activation='relu'), # bigger dense layer
    layers.Dropout(0.4),                 # NEW: prevent overfitting
    layers.Dense(10, activation='softmax')
])

new_CNN.compile(optimizer='adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])

```

Figure 5.2: new_CNN model

The training setup was also revised to support the enhanced model. The number of training epochs was increased from 10 to 15 to give the model more time to adjust its internal weights. The batch size was set explicitly to 64, which is a widely used value that balances training speed and model stability. Augmented images were supplied to the model using `datagen.flow()` to ensure that each batch contains freshly transformed images throughout training. The training code is as shown in Figure 5.3.

```

print('Training enhanced CNN (new_CNN)...')
start = time.time()
history_new = new_CNN.fit(
    datagen.flow(X_train, y_train, batch_size=64), # use augmented data
    epochs=15,
    validation_data=(X_test, y_test),
    verbose=1
)

```

Figure 5.3: new_CNN training code

6.0 Evaluation based on CNN and new_CNN model

Following the training of both models, a series of evaluations were carried out using the same 10,000 test images that neither model had been exposed to during training.

Firstly, the test accuracy of both models was measured using the `evaluate()` function as in Figure 6.1. This function computes the proportion of test images that were classified correctly.

```
# Get accuracy scores
_, cnn_acc = cnn.evaluate(x_test, y_test, verbose=0)
_, new_cnn_acc = new_CNN.evaluate(x_test, y_test, verbose=0)
```

Figure 6.1: Accuracy test using `evaluate()`

Figure 6.2 shows the enhanced model achieves an improvement of approximately 5 to 6 percentage points over the original CNN. This result indicates that the combined effect of data augmentation, deeper architecture and regularisation techniques leads to a meaningful gain in generalisation performance.

```
=== RESULTS ===
Original CNN accuracy : 70.32%
Enhanced CNN accuracy : 76.49%
Improvement           : +6.17%
```

Figure 6.2: `evaluate()` result

Second, Figure 6.3 presents a structured comparison of all architectural and training differences between the two models. As shown in Figure 6.3, every change made in `new_CNN` was targeted at a specific weakness of the original model. The addition of a third convolutional layer increases the model's representational depth. The 'BatchNormalization' and 'Dropout' improve training stability and reduce overfitting. Additionally, the larger Dense layer strengthens the classifier head while data augmentation and the extended training duration provide the model with greater exposure to varied examples.

Feature	CNN	new_CNN
Conv layers	2	3
BatchNormalization	No	Yes
Dropout	No	Yes (0.4)
Data Augmentation	No	Yes
Dense units	64	128
Epochs	10	15
Test Accuracy	70.32%	76.49%

Figure 6.3: Structural comparison between both architectures

Next, the `classification_report()` function was used to evaluate the performance of `new_CNN` on each of the 10 CIFAR-10 classes individually. Three metrics are reported for each class:

- Precision: The proportion of predictions for a given class that were correct. A high precision score indicates that the model rarely misidentifies other classes as this one.
- Recall: The proportion of actual instances of a class that were correctly identified by the model. A high recall score indicates that the model rarely misses true examples of this class.
- F1-Score: The harmonic mean of Precision and Recall. A score of 1.0 represents perfect classification; a score of 0.0 represents complete failure.

The classification report was generated using the code shown in Figure 6.4 and the results are in Figure 6.5.

```
y_pred_new = new_CNN.predict(X_test, verbose=0)
y_pred_classes = [np.argmax(p) for p in y_pred_new]

print('Classification Report - new_CNN')
print(classification_report(y_test, y_pred_classes, target_names=classes))
```

Figure 6.4: Classification report code

Classification Report – new_CNN				
	precision	recall	f1-score	support
airplane	0.82	0.75	0.78	1000
automobile	0.91	0.85	0.88	1000
bird	0.74	0.57	0.65	1000
cat	0.64	0.51	0.57	1000
deer	0.68	0.78	0.73	1000
dog	0.66	0.74	0.70	1000
frog	0.76	0.87	0.81	1000
horse	0.83	0.79	0.81	1000
ship	0.88	0.85	0.86	1000
truck	0.75	0.93	0.83	1000
accuracy			0.76	10000
macro avg	0.77	0.76	0.76	10000
weighted avg	0.77	0.76	0.76	10000

Figure 6.5: Classification report results

7.0 Results and Discussions

The experimental results reveal a clear and progressive improvement in classification performance across the three models evaluated in this study which are the Artificial Neural Network (ANN), the original Convolutional Neural Network (CNN), and the enhanced CNN (new_CNN). All models were evaluated on the same 10,000 test images from the CIFAR-10 dataset to ensure a fair and consistent comparison.

The ANN model, which relies solely on fully connected layers, achieved a test accuracy of approximately 45%. This limited performance can be attributed to its architectural design, which processes flattened pixel values and fails to exploit the spatial structure of images. As evidenced by the confusion matrix in Figure 3.7.2, the ANN exhibited significant misclassification, particularly among visually similar classes such as cats, dogs, and deer.

The original CNN model demonstrated a substantial improvement, achieving a test accuracy of 70.32%, as shown in Figure 3.10.1. This improvement of approximately 25 percentage points over the ANN confirms the effectiveness of convolutional and pooling layers in automatically extracting hierarchical spatial features from image data. The Conv2D layers successfully detected local patterns such as edges and textures, while the MaxPooling2D layers reduced spatial dimensions while retaining critical feature information.

The enhanced CNN model (new_CNN) further improved performance, achieving a test accuracy of 76.49%, representing an additional gain of 6.17 percentage points over the original CNN, as shown in Figure 6.2. This improvement can be attributed to the combined effect of data augmentation, the addition of a third convolutional block with 128 filters, BatchNormalization layers, Dropout regularisation, and an extended training duration of 15 epochs. As presented in the classification report in Figure 6.5, the new_CNN model performed consistently well across all 10 classes, with automobiles achieving the highest precision of 0.91 and cat remaining the most challenging class with a precision of 0.64, likely due to its visual similarity with dogs and deer.

Overall, these results demonstrate that systematic architectural enhancements and regularisation strategies lead to meaningful gains in generalisation performance on image classification tasks. However, a test accuracy of 76.49% still indicates room for further improvement, such as adopting deeper architectures or transfer learning approaches.

9.0 Reflections

Safiya Nursyahadah binti Masnoor

Throughout this tutorial, a deeper understanding was gained on how CNN outperforms ANN in image classification tasks, which was clearly reflected in the accuracy improvement from 45% to 70.32%. One challenge encountered was understanding why the ANN produced such low accuracy, which was resolved by studying how the Flatten layer destroys spatial relationships between pixels. In future, transfer learning using pre-trained models such as VGG16 or ResNet could be explored to further improve classification accuracy, as well as experimenting with a higher number of epochs to reduce overfitting and improve the model's generalisation on unseen data.

Farra Nurzahin binti Zaharil Anuar

Throughout this project, a better version of the CNN model was built and named new_CNN by adding a third convolutional layer with 128 filters, BatchNormalization after each convolutional layer, a Dropout layer with a rate of 0.4 to reduce overfitting and data augmentation to expose

the model to more image variety. One challenge faced was understanding how each change affected the model which was worked through by testing and observing the results. In the end, the new_CNN achieved a higher test accuracy than the original CNN which showed that small but thoughtful changes to a model's architecture can make a meaningful difference in performance.

Dayang Farah Farzana binti Abang Idham

Working on the foundational stage of this image classification project deepened the understanding of how proper data preprocessing, such as pixel normalization and label flattening, stabilizes gradient descent and successfully bridges the semantic gap. The initial implementation progressed smoothly without technical obstacles, as the environment setup, dataset loading, and sample visualization steps executed seamlessly exactly as outlined in the tutorial. The process successfully converted raw pixel intensities into normalized, floating-point inputs and flattened the target vectors into a clean 1D format for a completely error-free handoff to the training stage. For future data engineering applications, the workflow can be further enhanced by scripting automated exploratory data analysis (EDA) checks to dynamically audit the dataset for hidden imbalances before configuring the baseline network structure.

References

- Grandini, M., Bagli, E., & Visani, G. (2020). Metrics for multi-class classification: an overview. arXiv preprint arXiv:2008.05756.
- Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1412.6980>
- Krizhevsky, A., Nair, V., & Hinton, G. (2009). *CIFAR-10 and CIFAR-100 datasets*. University of Toronto. <https://www.cs.toronto.edu/~kriz/cifar.html>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. Nature, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>
- YouTube Tutorial (2024). *Image Classification Using CNN*. YouTube. <https://www.youtube.com/watch?v=Rmtr9SY-4VQ>